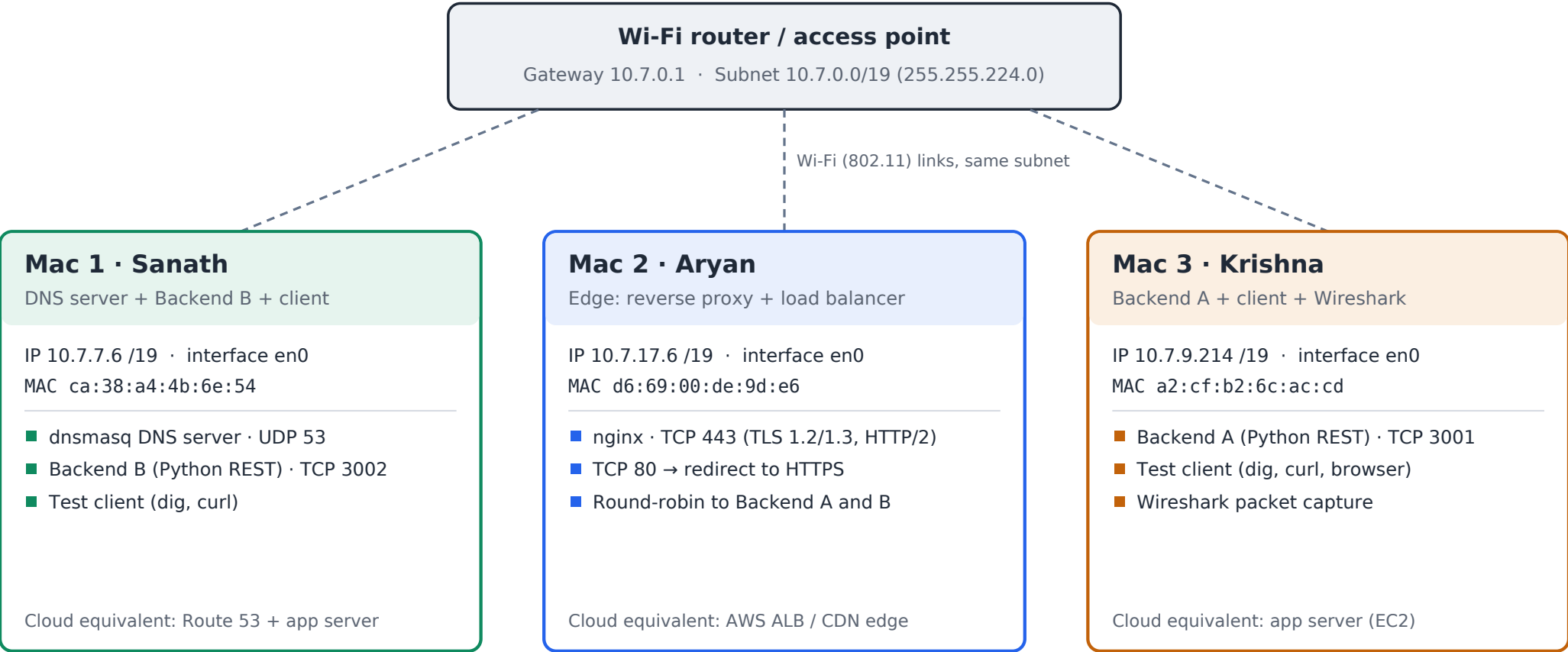


Private Network Service Platform — Phase 1 Architecture

Team: Sanath (Mac 1) · Aryan (Mac 2) · Krishna (Mac 3) · Private domain: team1.test · 5 October 2026

A client resolves **app.team1.test** through our own DNS server on Mac 1, connects over HTTPS to the nginx edge on Mac 2, which terminates TLS and load-balances each request round-robin across two Python REST backends (Backend A on Mac 3, Backend B on Mac 1). All three MacBooks share one Wi-Fi subnet, 10.7.0.0/19; nothing runs in the cloud.

1. Network topology



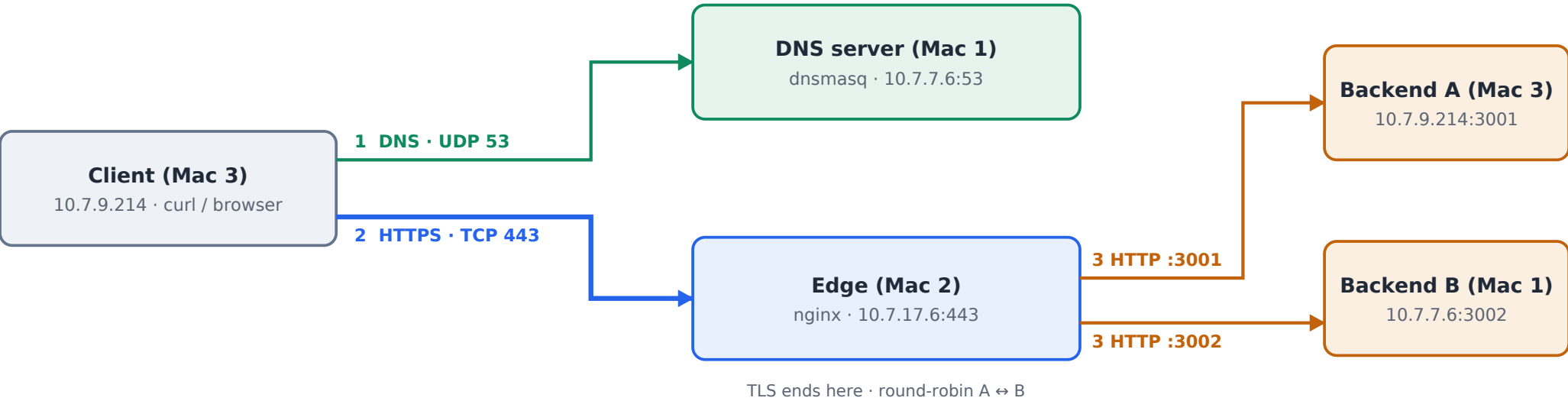
Physical topology: a star around the Wi-Fi access point. All three Macs are in the same /19 subnet (10.7.0.0 – 10.7.31.255), so they reach each other directly; the gateway 10.7.0.1 is only used for internet traffic.

2. Machine roles and IP / service table

Machine	Person	Role	Services (port)	IPv4	Subnet mask	Gateway	Interface	MAC address (in use)
Mac 1	Sanath	Private DNS server, Backend B, test client	dnsmasq (UDP 53), Backend B (TCP 3002)	10.7.7.6	255.255.224.0 (/19)	10.7.0.1	en0	ca:38:a4:4b:6e:54
Mac 2	Aryan	Edge: reverse proxy, load balancer, TLS termination	nginx (TCP 443 HTTPS, TCP 80 → redirect)	10.7.17.6	255.255.224.0 (/19)	10.7.0.1	en0	d6:69:00:de:9d:e6
Mac 3	Krishna	Backend A, test client, packet capture	Backend A (TCP 3001), Wireshark	10.7.9.214	255.255.224.0 (/19)	10.7.0.1	en0	a2:cf:b2:6c:ac:cd

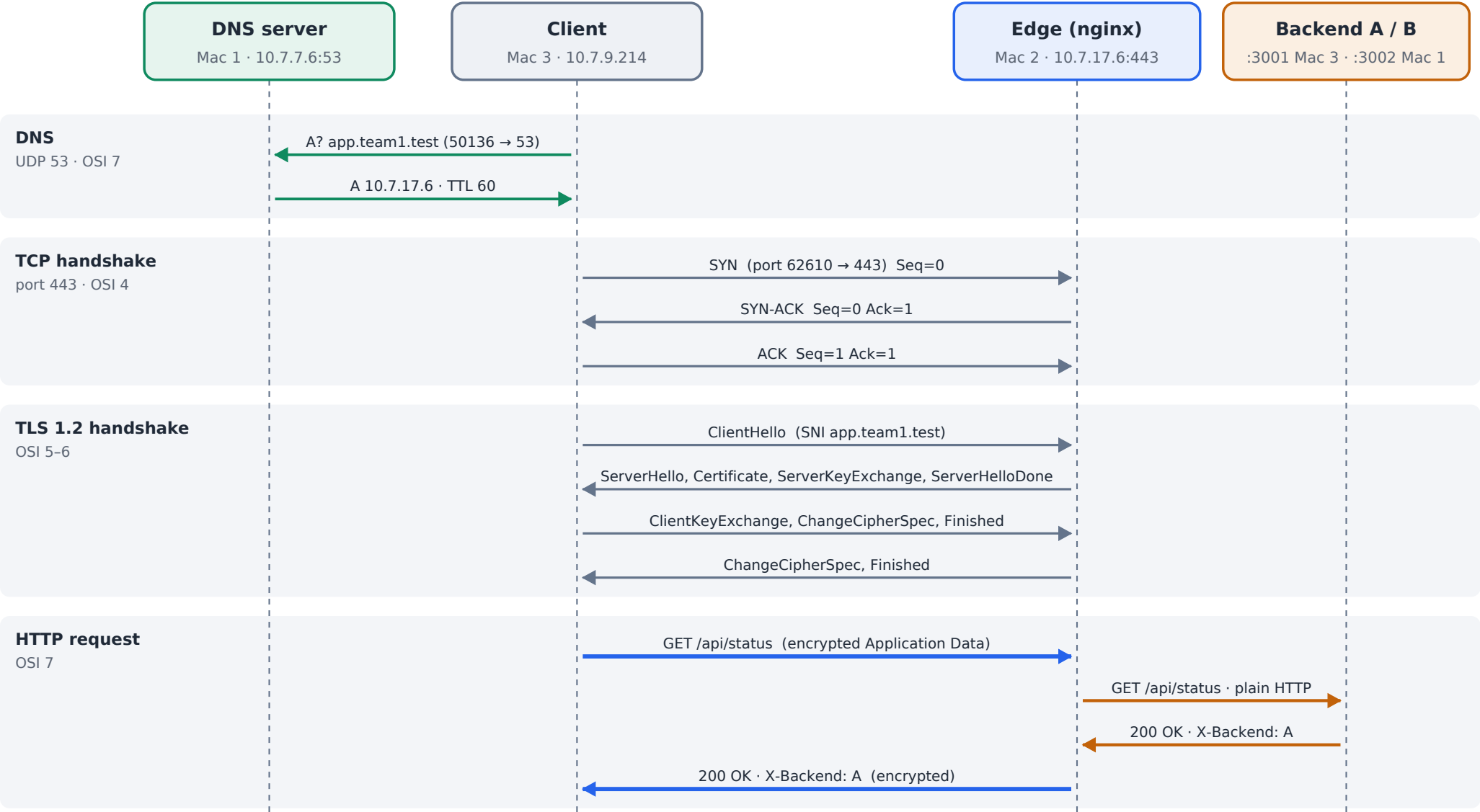
macOS uses a private Wi-Fi MAC per network; the addresses above are the ones seen on the LAN (and in Wireshark). Hardware MACs: 10:9f:41:c4:9d:1f (Mac 1), 10:9f:41:bb:9c:9c (Mac 2), 10:9f:41:bf:d0:70 (Mac 3).

3. Service map and request path



1 · The client asks Mac 1 for app.team1.test and gets 10.7.17.6 (Mac 2). 2 · It opens HTTPS to nginx on port 443. 3 · nginx decrypts the request and forwards it as plain HTTP to Backend A or B in turn. The client never knows the backend addresses.

4. Request flow by protocol layer — GET https://app.team1.test/api/status



Packet details from our capture phase1-flow.pcapng (filters: dns.qry.name == "app.team1.test", tcp.stream == 47, http). Blue = encrypted by TLS; orange = plain HTTP between the edge and a backend, because TLS ends at nginx.

5. Protocol layer mapping

Step	Protocol	Transport / port	OSI layer	TCP/IP layer	Where we show it
Find the server	DNS (A record)	UDP, client port → 53	7 Application	Application	dig / check-dns; Wireshark DNS
Open a connection	TCP 3-way handshake	TCP, client port → 443	4 Transport	Transport	Wireshark SYN / SYN-ACK / ACK
Secure the channel	TLS 1.2 / 1.3	inside TCP 443	5-6 Session / Presentation	Application	Wireshark TLS handshake + certificate
Ask for data	HTTP/2 or HTTP/1.1 (REST)	inside TLS	7 Application	Application	curl -v headers; X-Backend
Address the hosts	IPv4, subnet 10.7.0.0/19	—	3 Network	Internet	IP header in Wireshark
Deliver on the LAN	Wi-Fi 802.11 (Ethernet II frames)	MAC addresses	2 Data link / 1 Physical	Link	Ethernet II header in Wireshark

6. Configuration summary

Component	Machine	Configuration
DNS (dnsmasq)	Mac 1	app.team1.test → A 10.7.17.6 and api.team1.test → A 10.7.17.6, TTL 60 s. Other names forwarded to 1.1.1.1 / 8.8.8.8. Mac 2 and Mac 3 use 10.7.7.6 as their DNS server; Mac 1 uses itself (127.0.0.1).
Edge (nginx 1.31.6)	Mac 2	listen 443 ssl + http2; port 80 redirects to HTTPS. upstream (round-robin): 10.7.9.214:3001 (A), 10.7.7.6:3002 (B). Passive failover: max_fails=1, fail_timeout=10s, proxy_next_upstream error timeout, proxy_connect_timeout 2s.
TLS	Mac 2 + all clients	Own CA "team1 Local Root CA" (RSA 2048), trusted in each Mac's System keychain. Server certificate CN app.team1.test, SAN app.team1.test + api.team1.test, valid 4 Oct 2026 – 4 Oct 2027. TLS 1.2 and 1.3 enabled.
Backends (Python)	Mac 3 (A), Mac 1 (B)	GET / (page), GET /api/status (JSON with backend id, Cache-Control: no-store), GET /api/cached (Cache-Control: public, max-age=60 + ETag → 304). Every response carries X-Backend: A or B. Bound to 0.0.0.0.

7. Cloud equivalents

Our component	Cloud equivalent	Shared job
dnsmasq on Mac 1	AWS Route 53 private hosted zone	Maps private names to IP addresses
nginx on Mac 2	AWS Application Load Balancer / CDN edge	Single entry point, TLS termination, load balancing, health checks
Backends on Mac 3 and Mac 1	EC2 instances in a target group	Identical app servers behind the load balancer
Our local CA	AWS Certificate Manager / private CA	Issues the TLS certificate
Wi-Fi LAN 10.7.0.0/19	VPC subnet	Private network shared by the machines